

A PRACTITIONER'S TEXTBOOK IN APPLIED CRYPTOGRAPHY

Protocols and Operations

Volume 2

Certificates, keys, secrets and hardware roots of trust, from
first principles to production



CONTENTS

What is in this draft

This draft contains Chapter 22. The remaining chapters of Volume 2 follow the outline below and will be added as they are written.

PART V · CAPSTONE: ENTERPRISE CERTIFICATE AUTOMATION

22	Running a Certificate Lifecycle Platform	3
•	Before you begin	4
22.1	Why enterprises buy a certificate lifecycle platform	8
22.2	Reference architecture	12
22.3	Discovery and inventory	17
22.4	Policy and governance	21
22.5	CA integrations: DigiCert, Sectigo and dual-CA failover	25
22.6	Provisioning models and private key custody	30
22.7	Identity, access and approvals	34
22.8	Renewal, rotation and revocation	37
22.9	The rollout program and the metrics that prove it works	42
•	Lab: a miniature certificate platform on your laptop	46
•	At work: checklists and templates to copy	52
•	Failure story: the expired certificate behind the Equifax breach	56
•	Chapter summary	58
•	Review questions	59
•	Sources	60

ALSO IN VOLUME 2 (IN PROGRESS)

Part I	Secure channels	Chapters 1 to 6
Part II	PKI	Chapters 7 to 12
Part III	Keys and secrets	Chapters 13 to 17
Part IV	Hardware and architecture	Chapters 18 to 21
Part V	Capstone: enterprise certificate automation	Chapters 22 to 23



Running a Certificate Lifecycle Platform

This chapter teaches you how a certificate lifecycle platform such as CyberArk Certificate Manager works, why it exists, and how to run one well. By the end you should be able to explain every moving part of your own rollout and ask the questions that prevent outages.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain the four layers of a certificate platform and where private keys should live.
- Plan discovery, policy, CA integration and provisioning for a real estate.
- Design dual-CA failover with DigiCert and Sectigo, including DNS validation and CAA.
- Run safe renewals and revocations, and measure a rollout with the right metrics.

IN THIS CHAPTER

- | | |
|------|---|
| 22.1 | Why enterprises buy a certificate lifecycle platform |
| 22.2 | Reference architecture |
| 22.3 | Discovery and inventory |
| 22.4 | Policy and governance |
| 22.5 | CA integrations: DigiCert, Sectigo and dual-CA failover |
| 22.6 | Provisioning models and private key custody |
| 22.7 | Identity, access and approvals |
| 22.8 | Renewal, rotation and revocation |
| 22.9 | The rollout program and the metrics that prove it works |

Before you start: no earlier chapter is required. Every term is defined in plain words in the first section.

START HERE

Before you begin

What this chapter covers

A large company has thousands, sometimes hundreds of thousands, of TLS certificates. They sit on load balancers, firewalls, content delivery networks, Kubernetes clusters, and servers nobody remembers building. Each one expires on a fixed date. When one expires without being replaced, the service that uses it stops working for its users.

A certificate lifecycle platform is software that finds all of those certificates, decides what rules they must follow, gets new ones from a certificate authority, and installs them in the right place before the old ones expire. This chapter walks through each of those jobs in the order you would meet them in a real rollout.

What you should already know

You do not need to have read the earlier chapters. Every term this chapter uses is defined below in plain words. If a definition feels thin, the chapter number in brackets points to where it is covered in depth.

The terms, in plain words

Read this list once now. You can come back to it whenever a term is unclear.

Key pair.

Two mathematically linked numbers created together. One is the **private key**, which must stay secret. The other is the **public key**, which can be shared with anyone. Something locked with one can only be unlocked or checked with the other. (Volume 1)

Private key.

The secret half of a key pair. Whoever holds it can prove they are the owner of a certificate. If an attacker copies it, they can pretend to be your website. Protecting private keys is the most important job in this whole chapter.

Certificate.

A small signed file that says "this public key belongs to this name" (for example, `www.example.com`). It is like a passport: the passport office (the certificate authority) vouches that the photo (the public key) belongs to the person named. A certificate is public. It is useless to an attacker without the matching private key. (Chapter 7)

Certificate authority (CA).

The organization or system that signs certificates. **Public CAs**, such as DigiCert and Sectigo, are trusted by every web browser and operating system. **Private CAs** are run by a company for its own internal systems and are trusted only by machines the company configures to trust them. (Chapter 8)

Root and intermediate certificates.

A CA does not sign your certificate directly with its most precious key. It keeps a **root** key offline and uses it to sign an **intermediate** certificate. The intermediate signs your certificate. The **chain** is your certificate plus the intermediates needed to link it back to the root. Servers must send the chain, or some clients will reject the connection. (Chapter 9)

Certificate signing request (CSR).

A file you send to a CA to ask for a certificate. It contains your public key and the names you want, and it is signed with your private key to prove you hold it. The private key itself never leaves your system in a CSR.

Subject Alternative Name (SAN).

The list of names a certificate is valid for, such as `example.com` and `www.example.com`. Browsers check this list, not the older "Common Name" field.

Validity period.

The dates between which a certificate is valid: **Not Before** and **Not After**. After Not After, the certificate is **expired** and clients refuse it.

Renewal.

Getting a new certificate for the same names before the old one expires. A good renewal also creates a new key pair. Reusing the old key is allowed but weaker, because a key that leaked months ago would still work.

Rotation.

Replacing a key or certificate on a schedule, whether or not it is about to expire.

Revocation.

Telling the world a certificate must no longer be trusted before its expiry date, usually because its private key leaked. The CA publishes this through lists (CRLs) or an online check (OCSP). (Chapter 9)

Domain validation (DV), organization validation (OV), extended validation (EV).

How much the CA checks before issuing. DV proves you control the domain. OV also checks that your company exists. EV checks more deeply. Most automation uses DV or OV.

Machine identity.

Industry term for any credential a machine uses to prove who it is: TLS certificates, SSH keys, code signing certificates, and similar. CyberArk uses this phrase often.

Certificate lifecycle management (CLM).

The full process of discovering, requesting, issuing, installing, monitoring, renewing and revoking certificates. A **CLM platform** is software that does this centrally. CyberArk Certificate Manager is a CLM platform.

Target, or endpoint.

The place a certificate is installed and used: an F5 load balancer, an Akamai edge configuration, a Palo Alto firewall, a Kubernetes cluster, a web server.

Connector, driver, or agent.

The piece of software that lets the platform talk to a target. A **connector** usually runs centrally and calls the target's management API (for example, F5's REST API). An **agent** is installed on the target machine itself and acts locally. Vendors use these words loosely; this chapter uses them as defined here.

ACME.

Automatic Certificate Management Environment, a standard protocol (RFC 8555) that lets software request and renew certificates with no human involved. Let's Encrypt made it popular. DigiCert and Sectigo both offer ACME endpoints. (Chapter 12)

Inventory.

The platform's database of every certificate it knows about: where it lives, who owns it, when it expires, and what policy it should follow.

Policy.

The rules certificates must follow, such as "RSA keys must be at least 2048 bits" or "only names under `example.com` are allowed".

■ How to read this chapter

Sections 22.1 to 22.9 follow the order of a real rollout: why, what it looks like, find everything, set the rules, connect the CAs, install certificates, control who can do what, keep certificates fresh, and run the program. Each section ends with a short **In your rollout** box that turns the section into questions you can take to your own project.

22.1

Why enterprises buy a certificate lifecycle platform

Companies buy these platforms for four reasons: expired certificates cause outages, nobody knows how many certificates exist, certificate lifetimes are getting much shorter, and sometimes a whole batch of certificates must be replaced in days. Each reason is explained below with a real example.

A note on product names

The product your team is rolling out has changed names several times, and you will hear all of these names in meetings. They refer to the same product family:

Name you may hear	What it is
Venafi Trust Protection Platform (TPP), Venafi TLS Protect, TLS Protect Datacenter	The older names of the version you install and run in your own data center
CyberArk Certificate Manager, Self-Hosted	The current name of that installed version, after CyberArk bought Venafi in 2024
Venafi as a Service (VaaS), TLS Protect Cloud	The older names of the cloud-hosted version
CyberArk Certificate Manager, SaaS	The current CyberArk name of the cloud-hosted version
Palo Alto Networks Next-Gen Trust Security (NGTS)	Palo Alto Networks completed its purchase of CyberArk in February 2026 and now markets the SaaS certificate manager under this name

So when someone says "Venafi", "TPP", "CyberArk Certificate Manager" or "NGTS", they are all talking about this platform. The Self-Hosted and SaaS versions do the same job but use different words for some parts. This chapter gives both names where they differ.

The Palo Alto Networks purchase matters for your project in one practical way: Palo Alto firewalls and Panorama are now made by the same company as your certificate platform. Expect that integration to get closer attention from the vendor over time. Check current product documentation rather than assuming, because naming and packaging are still settling.

Reason 1: an expired certificate takes a service down

When a certificate expires, every client that checks it (browsers, mobile apps, other servers, payment partners) refuses to connect. There is no grace period. The service is down until someone installs a new certificate. Three well-known examples:

- **Ericsson and O2, December 2018.** An expired certificate inside Ericsson software running in mobile networks caused a data outage for millions of O2 customers in the UK and customers of other carriers, for most of a day.
- **Microsoft Teams, February 2020.** An authentication certificate expired and Teams was unavailable for a few hours for users around the world.
- **Equifax, 2017.** A certificate on a network monitoring device that inspected encrypted traffic had expired 19 months earlier. While it was expired, the device could not inspect traffic, so the attackers' data theft went unnoticed for 76 days. Here the expired certificate did not cause an outage. It quietly switched off a security control.

In each case the certificate was not hard to replace. The problem was that nobody responsible knew it was about to expire. That is a tracking problem, and tracking is the first thing a platform solves.

Reason 2: nobody knows how many certificates exist

Certificates are created by many teams for many reasons. A developer gets one for a test server. A network engineer installs one on a firewall. A vendor appliance generates its own. A cloud team requests one from a different CA with a company credit card. None of these is recorded in one place.

Certificates nobody is tracking are called **shadow certificates** or **rogue certificates**. They are dangerous for three reasons:

1. Nobody renews them, so they expire and cause outages.
2. Nobody checks them, so they may use weak keys or come from CAs the company does not approve.
3. Nobody owns them, so when a private key leaks, nobody knows which systems to fix.

When companies run their first full discovery scan, they commonly find several times more certificates than they believed they had. Section 22.3 explains how discovery works.

Reason 3: certificate lifetimes are getting much shorter

This is the reason that turns certificate automation from "good practice" into "required".

The **CA/Browser Forum** is the group of certificate authorities and browser makers (Apple, Google, Microsoft, Mozilla and others) that sets the rules for public certificates. In April 2025 it passed Ballot SC-081, which cuts the maximum lifetime of public TLS certificates in three steps:

Certificates issued on or after	Maximum lifetime	Maximum reuse of a domain ownership check
Before 15 March 2026	398 days	398 days
15 March 2026 (in force now)	200 days	200 days
15 March 2027	100 days	100 days
15 March 2029	47 days	10 days

Two things in this table need explaining.

First, these rules apply only to **public** certificates, the ones from DigiCert, Sectigo and other CAs that browsers trust. Certificates from your own private CA are not bound by them, although many companies choose to follow similar limits.

Second, the last column matters as much as the lifetime. Before a public CA issues a certificate, it must check that you control the domain (for example, by asking you to place a value in DNS). Today it can reuse that check for months. From 2029 it can reuse it for only 10 days. In practice this means nearly every renewal will need a fresh domain check, so the domain check itself has to be automated too, usually through DNS.

Now look at what shorter lifetimes do to workload. Teams usually renew when about a third of the lifetime remains, so they are never close to the edge. For a company with 1,000 public certificates, that gives roughly:

Maximum lifetime	Renew every (about)	Renewals per year for 1,000 certificates
398 days	265 days	1,400
200 days	133 days	2,700
100 days	67 days	5,500
47 days	31 days	11,800

At 398 days, a team can survive on spreadsheets and calendar reminders. At 47 days, the same team would be renewing about 45 certificates every working day. Manual work cannot keep up, and every manual step is a chance for a mistake. That is why your company is investing in automation now, while the limit is still 200 days.

Reason 4: sometimes everything must be replaced at once

Most of the time, certificates are replaced one at a time on their own schedule. Occasionally, a large group must be replaced within days:

- **DigiCert, July 2024.** DigiCert found a bug in one of its domain validation methods. Under CA/Browser Forum rules it had to revoke about 83,000 affected certificates within 24 hours (later extended for some critical customers). Companies with automation renewed in minutes. Companies without it spent days in emergency calls.
- **Entrust, 2024.** Google Chrome and other browsers announced they would stop trusting new Entrust public certificates after compliance failures. Every Entrust customer had to move to a different CA.
- **Post-quantum migration, coming.** In the next few years, the algorithms inside certificates will change to resist quantum computers (Volume 3). That will mean replacing almost every certificate and many CAs.

A platform lets you answer "which certificates came from this CA, and where are they installed?" in seconds, and then replace them in bulk. This ability is called **crypto agility**: being able to change CAs, algorithms or keys quickly without breaking things.

■ What a platform actually does

Putting the four reasons together, a certificate lifecycle platform has six jobs:

1. **Discover** every certificate, wherever it is (section 22.3).
2. **Govern** them with policy: which CAs, key types, names and lifetimes are allowed (22.4).
3. **Issue** new certificates by talking to CAs such as DigiCert and Sectigo (22.5).
4. **Provision**, meaning install the certificate and key on the target and make sure the target uses it (22.6).
5. **Control access**: who may request, approve and see what (22.7).
6. **Renew and revoke** automatically and safely (22.8).

The rest of this chapter takes these jobs one at a time.

IN YOUR ROLLOUT

- ▶ How many certificates did your first discovery find, compared with what the business expected?
- ▶ Which public certificates will hit the 100-day limit in March 2027, and which targets cannot yet renew without a human?
- ▶ Is your DNS set up so domain checks can be automated for DigiCert and Sectigo?
- ▶ If DigiCert or Sectigo had to revoke a batch of your certificates within 24 hours, could you replace them in time?

22.2 Reference architecture

Every certificate lifecycle platform has the same four layers: the certificate authorities that sign certificates, the platform that holds the inventory and the rules, a connector runtime that does work inside your network, and the targets where certificates are actually used. Once you can picture these four layers, every conversation about the rollout becomes easier to follow.

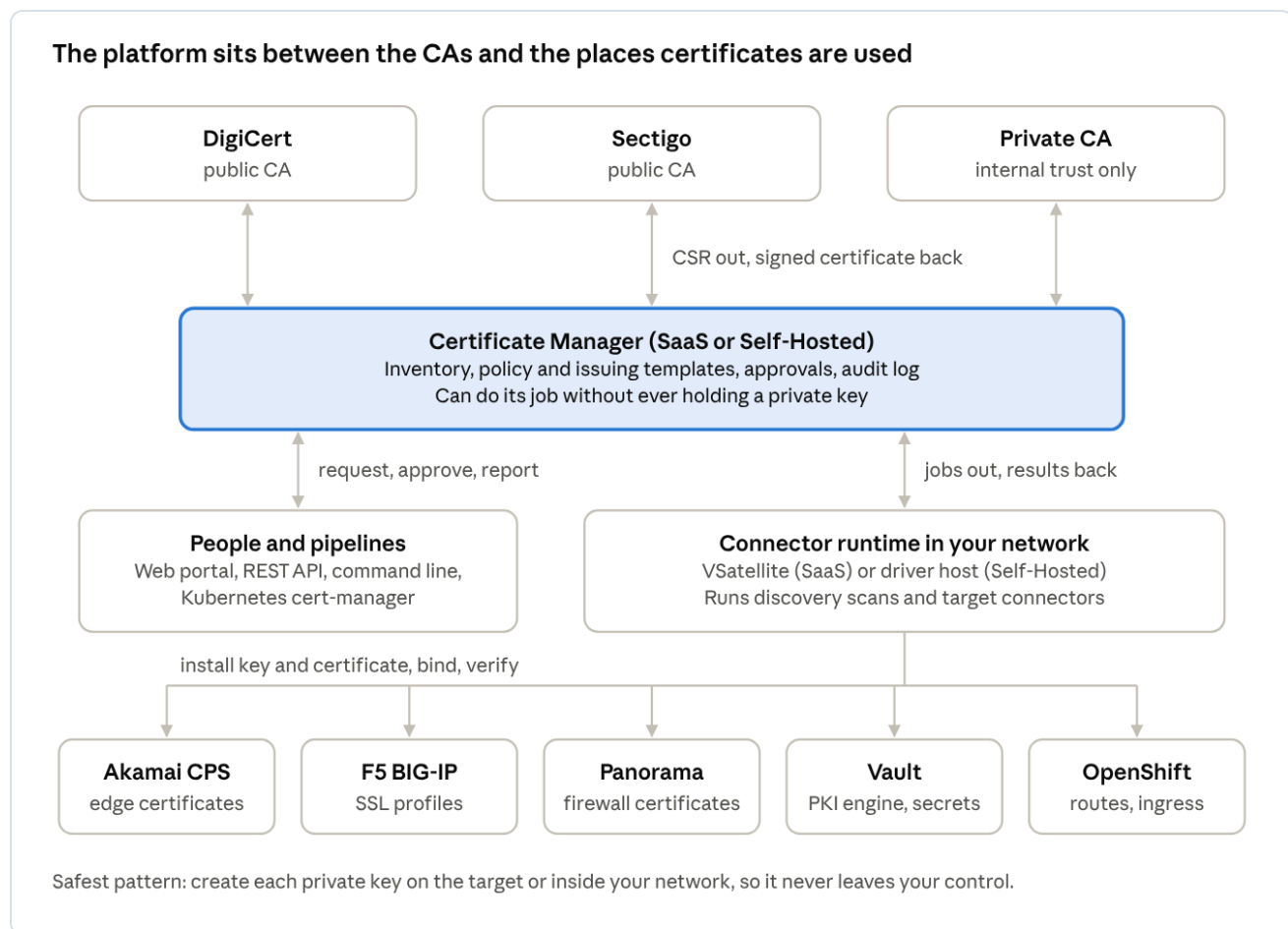


Figure 22.1 The reference architecture: certificate authorities at the top, the platform in the middle, a connector runtime inside your network, and the targets where certificates are used.

Read the picture from top to bottom. The platform never talks to a firewall or load balancer directly from the internet. It hands jobs to the connector runtime inside your network, and the runtime talks to the targets.

Layer 1: the certificate authorities

At the top are the CAs that sign certificates. In your company these are **DigiCert** and **Sectigo** for public certificates, and possibly a private CA (such as Microsoft Active Directory Certificate Services, or HashiCorp Vault's PKI engine) for internal ones. The platform connects to each CA through that CA's API. It sends a certificate signing request (CSR) and receives the signed certificate back. Section 22.5 covers this connection in detail.

The platform is **not** a CA itself in this picture. It is a broker: it asks CAs to sign, and keeps track of what they signed. (Some platforms can also act as a simple private CA, but that is a separate feature.)

Layer 2: the platform, or control plane

The **control plane** is the central brain. It is the part you log in to. It stores:

- **The inventory:** every certificate it knows about, with its expiry date, owner, issuing CA, and every place it is installed.
- **The policy:** the rules certificates must follow (section 22.4).
- **The workflow:** who asked for what, who approved it, and what happened (section 22.7).
- **The audit log:** an unchangeable history of every action, which auditors will ask for.

The control plane comes in two forms, and the words differ between them. This table maps them, so you can follow conversations in either:

Idea	Self-Hosted (formerly TPP)	SaaS (formerly TLS Protect Cloud, now also NGTS)
Where it runs	Your own Windows servers and database, in your data center	Run by the vendor in the cloud
How certificates are grouped	Folders in the Policy Tree	Applications
Where the rules live	Policies set on folders, inherited by everything below	Issuing templates assigned to applications
How it connects to a CA	CA template objects	CA connectors (CA accounts)
How a target is represented	A Device object (the box) with Application objects under it (each place a certificate is used on that box)	Machines and machine identities , handled by machine connectors
Code that talks to targets	Drivers from the driver library, or Adaptable drivers you write in PowerShell when no built-in driver exists	Machine connectors, run inside your network by a VSatellite

Idea	Self-Hosted (formerly TPP)	SaaS (formerly TLS Protect Cloud, now also NGTS)
Who can do what	Permissions on Policy Tree folders	Teams given access to applications and issuing templates

Notice one trap in the words: in Self-Hosted, an "Application" object means **one installed copy of a certificate on a device**. In SaaS, an "Application" means **a business application that owns a group of certificates**. When someone says "application" in a meeting, ask which one they mean.

Layer 3: the connector runtime in your network

Your firewalls, load balancers and clusters are inside private networks that the internet cannot reach, and that is correct. So the platform needs a helper inside your network.

- In **SaaS**, this helper is the **VSatellite**: a small, self-updating service (built on Kubernetes, running on a Linux machine you provide) that sits inside your network. It reaches out to the cloud platform to collect jobs, so you do not have to open inbound firewall ports. It runs discovery scans and the machine connectors that install certificates.
- In **Self-Hosted**, the platform servers are already in your network. The work is done by the platform's own services and drivers, sometimes on separate servers placed near the targets.

The connector runtime needs network access and credentials for every target it manages. That makes it a sensitive system. If an attacker controlled it, they could install their own certificates on your load balancers. Treat it with the same care as a domain controller.

Layer 4: the targets

At the bottom are the systems that use certificates: Akamai, F5 BIG-IP, Palo Alto Panorama, HashiCorp Vault and OpenShift in your case. Each one stores certificates differently and has its own API, so each needs its own connector. Chapter 23 gives a full playbook for each of your five targets.

Installing a certificate on a target is more than copying a file. A complete installation has three steps:

1. **Install**: put the new certificate, its chain and (if needed) its private key on the target.
2. **Bind**: tell the target to use the new certificate, for example by updating an F5 SSL profile or a Palo Alto SSL/TLS service profile. Until it is bound, the target keeps serving the old one.

- 3. Verify:** connect to the service and confirm it now presents the new certificate. Without this step, the platform only knows it sent a certificate, not that users are getting it.

People and pipelines

On the left of the picture are the humans and automated systems that use the platform:

- **The web portal** for PKI administrators and application owners.
- **The REST API**, for scripts and other tools.
- **Command-line tools and libraries**, such as CyberArk's VCert, used inside build pipelines.
- **Kubernetes cert-manager** with a CyberArk issuer, so clusters can request certificates that follow company policy (Chapter 23 covers this for OpenShift).

Where private keys live: the most important design decision

Every certificate has a private key, and the private key is the secret. The architecture has three options for where it is created:

Option	How it works	When to use it	Risk
On the target (best)	The target creates the key and a CSR. The platform sends the CSR to the CA and installs the returned certificate. The key never leaves the target	Targets that can create CSRs through their API, such as F5 and Akamai CPS	Lowest. Only the target ever holds the key
In your network, by the connector runtime or platform	The platform or VSatellite creates the key, gets the certificate, then pushes key and certificate to the target	Targets that cannot create CSRs, high-availability pairs that need the same key, or when keys must be backed up	Medium. The key passes over your network and may be stored by the platform, so storage must be encrypted, ideally in an HSM
By the CA	The CA creates the key and sends it to you	Avoid for TLS	Highest. A third party has had your private key

When someone proposes a new integration, the first question to ask is: **where is the private key created, and who can read it afterwards?** Section 22.6 goes deeper.

One renewal, step by step

To tie the layers together, here is what happens when an F5 certificate approaches expiry:

1. The control plane sees that the certificate has reached its renewal point (for example, 30 percent of its life left).

2. It checks the policy for that certificate's folder or application: which CA, which key type, which names are allowed.
3. It sends a job to the connector runtime: "renew this certificate on this F5".
4. The runtime asks the F5, through its API, to create a new key and CSR. The key stays on the F5.
5. The runtime passes the CSR to the control plane, which sends it to DigiCert through the CA connector.
6. DigiCert checks domain control and returns the signed certificate.
7. The runtime installs the certificate and chain on the F5, binds it to the SSL profile, and syncs the change to the F5's partner device.
8. The runtime connects to the website and checks that the new certificate is being served.
9. The control plane updates the inventory and writes every step to the audit log.

If any step fails, the platform should alert the certificate's owner and keep retrying, while the old certificate stays in place. Section 22.8 covers failures.

IN YOUR ROLLOUT

- ▶ Are you on SaaS (with VSatellites) or Self-Hosted? Which words does your team use?
- ▶ For each of your five targets, is the private key created on the target or pushed by the platform?
- ▶ Where does the platform store private keys it holds, and is that storage backed by an HSM (for example, Thales Luna)?
- ▶ Who can log in to the VSatellite or driver servers, and are they monitored like other critical systems?
- ▶ Does every integration include the verify step, or does it stop at install?

22.3 Discovery and inventory

You cannot renew a certificate you do not know exists. Discovery is how the platform finds certificates, and the inventory is where it records them. No single discovery method finds everything, so a good rollout combines several and then does the hard work of assigning an owner to each certificate.

The five ways to find certificates

1. Network scanning.

The connector runtime connects to a range of IP addresses and ports and starts a TLS handshake with each one, the same way a browser would. If something answers with a certificate, the scanner records it, along with where it was found (IP address, port, and hostname). This finds certificates that are actually being served, including on systems nobody told you about.

2. Integration discovery.

The platform logs in to a system's management API and reads its certificate store directly. For example, it can list every certificate on an F5, every enrollment in Akamai CPS, every certificate in a Panorama template, or every TLS secret in an OpenShift cluster. This finds certificates that are stored but not currently served, and it shows exactly which profile or route uses each one.

3. CA account import.

The platform connects to your DigiCert and Sectigo accounts and pulls the list of every certificate they have issued to your company. This shows what was bought and by whom, but not where each certificate was installed.

4. Certificate Transparency monitoring.

Every public CA must publish every certificate it issues to public **Certificate Transparency (CT) logs** (Chapter 10). By watching these logs for your domain names, you see every public certificate issued for your domains by **any** CA, including CAs your company did not approve. This is how you catch a team that bought a certificate elsewhere, or an attacker who tricked a CA.

5. Host agents or file scanning.

Software on a server searches its disk for certificate and key files (PEM files, Java keystores, Windows certificate stores). This finds certificates used by internal applications that never appear on a network port you scanned.

The table shows what each method finds and what it misses:

Method	Finds	Misses
Network scanning	Anything served on a scanned IP and port, including unknown systems	Ports and networks you did not scan, certificates not currently served, sites that need a hostname to answer (see SNI below)
Integration discovery	Everything stored on a connected system, plus how it is bound	Systems you have not connected
CA account import	Everything bought through your company CA accounts	Where certificates are installed, certificates from other CAs
Certificate Transparency	Every public certificate for your domains, from any CA	Private certificates, where they are installed
Host agents or file scanning	Certificate and key files on disk, including unused ones	Hosts without the agent

In practice, start with integration discovery on the five platforms you are already connecting (Akamai, F5, Panorama, Vault, OpenShift), add CA import from DigiCert and Sectigo, turn on CT monitoring, and then use network scanning to find what all of those missed.

Three things that make network scanning harder than it looks

Server Name Indication (SNI).

One IP address often hosts many websites, each with its own certificate. During the handshake, the client says which site it wants by sending the hostname in a field called **SNI**. If the scanner connects with no hostname, the server returns only its default certificate, and every other certificate on that IP stays hidden. Good scans therefore feed the scanner a list of hostnames as well as IP ranges, often taken from DNS zones.

Non-standard ports.

TLS is not only on port 443. Common other places include 8443 and other web ports, 636 (LDAP over TLS), 993 and 995 (mail), 5671 (AMQP messaging), and 9093 (often Kafka). Mail servers on ports 25 and 587 start in plain text and switch to TLS with a command called STARTTLS, which the scanner must support. Agree with each team which ports their services use.

Scanning looks like an attack.

A discovery scan is technically a port scan. Before running one, tell your security operations team, register the scanner's IP addresses, and agree on rate limits so fragile devices are not overwhelmed. Place a scanner (a VSatellite in SaaS) in each network zone, because firewalls between zones will block cross-zone scans.

From findings to an inventory

Raw discovery results are messy. The same certificate shows up many times, many entries are test or vendor certificates, and almost none come with an owner. Turning findings into a useful inventory takes four steps.

Step 1: deduplicate.

A certificate is identified by its **fingerprint** (a hash of the whole certificate). If the same fingerprint is found on four F5 virtual servers and in a DigiCert account, that is **one certificate** with **four installations**. The inventory must keep these separate, because renewal creates one new certificate but must update all four installations.

Step 2: classify.

For each certificate, record: public or private, which CA issued it, key type and size, signature algorithm, the names it covers, its expiry date, and whether it is self-signed. Classification lets you spot problems in bulk, for example every certificate still using a 1024-bit RSA key or SHA-1.

Step 3: assign an owner.

This is the hardest and most important step. An owner is a person or team who will be alerted and will act when something goes wrong. Common ways to work out ownership:

- Match the certificate's names (for example `payments.example.com`) to your application catalogue or configuration management database (CMDB).
- Use the requester recorded in the DigiCert or Sectigo account.
- Use the owner of the system it was found on (the F5 team, the OpenShift cluster owner).
- Ask. Anything still unknown goes into an **unowned** queue that a person works through each week.

Step 4: decide what happens next.

Each certificate gets one of three outcomes: **manage** (bring it under the platform and automate it), **monitor** (watch its expiry but do not automate yet, for example because no connector exists), or **retire** (it is no longer needed; remove it).

What the inventory stores

An inventory has two kinds of record. Keep the difference in mind, because many reporting mistakes come from mixing them up.

Record	One per	Key fields
Certificate	Unique certificate (fingerprint)	Subject and SAN names, issuer, serial number, key type and size, Not Before and Not After, fingerprint, owner, policy or application it belongs to
Installation	Each place the certificate is used	Target system, IP and port or API location, which profile or route uses it, how it was found, when it was last seen, whether the platform can automate it

When someone asks "how many certificates do we have?", check whether they mean unique certificates or installations. The numbers can differ by a factor of five or more.

Keeping the inventory true

An inventory is only useful while it matches reality. Schedule discovery to repeat (daily for integrations and CT, weekly for network scans is a reasonable start). Mark installations that have not been seen for a set period as possibly removed, and confirm before deleting them. Track a few numbers over time:

- Percentage of certificates with a named owner.
- Number of certificates in the unowned queue, which should fall every month.
- Percentage of certificates that are **managed** rather than only **monitored**.
- Number of certificates found in CT logs that were not issued through the platform. Each one is either a gap in your process or a security problem.

IN YOUR ROLLOUT

- ▶ Which discovery methods are switched on today, and which of the five methods are missing?
- ▶ Do network scans send hostnames (SNI), or only IP addresses?
- ▶ Is there a VSatellite or scanner in every network zone that holds certificates?
- ▶ Who works the unowned queue, and how fast is it shrinking?
- ▶ Are your dashboards counting certificates, installations, or a mix of both?

22.4 Policy and governance

Policy is the set of rules every certificate must follow. The platform uses it in two ways: to **prevent** bad certificates by refusing requests that break the rules, and to **detect** bad certificates that already exist by comparing discovered certificates against the rules. Governance is the human side: who writes the rules, who can change them, and how exceptions are handled.

What a certificate policy controls

Each row below is a decision your PKI team makes once and the platform then enforces every time.

Setting	What it means	Typical choice for public TLS
Allowed CAs	Which CA accounts may sign certificates for this group	DigiCert as primary, Sectigo as backup
Key algorithm and size	The kind of key pair. RSA uses large numbers; ECDSA uses elliptic curves and gives the same security with much smaller keys	ECDSA P-256, or RSA 2048 or 3072 where older clients need RSA
Allowed domains	Which names may appear in certificates	Only names under domains the company owns, such as <code>example.com</code>
Wildcards	Whether names like <code>*.example.com</code> are allowed	Blocked by default, allowed only with approval
Maximum validity	The longest lifetime allowed	199 days for public certificates today, since the public limit is 200 days
Key reuse	Whether a renewal may keep the old private key	Not allowed: every renewal creates a new key
Key generation location	Where the private key must be created (section 22.2)	On the target where possible
Subject fields	Organization, country and similar fields shown in the certificate	Fixed by the template, so nobody can type them wrong
Extended key usage (EKU)	What the certificate may be used for: server authentication, client authentication, code signing	Server authentication only for public TLS

Some of these deserve a closer look.

Why block wildcards by default.

A wildcard certificate for `*.example.com` is valid for every name one level under `example.com`. The same private key is usually copied to many servers. If any one of those servers is broken into, the attacker can impersonate every site the wildcard covers. Wildcards are convenient, but each one makes the damage from a single break-in much larger.

Why public TLS certificates should be server authentication only.

Browsers now require public CAs to use their TLS hierarchies only for websites. Public CAs, including DigiCert and Sectigo, have been removing the "client authentication" usage from public TLS certificates. If any of your systems use a public certificate to authenticate as a **client** (for example, a server connecting to a partner with mutual TLS), those connections need a different certificate type or a private CA. Sectigo stopped including client authentication in its public TLS certificates on 15 May 2026. DigiCert's change moved to around March 2027 after Chrome extended its timeline. Confirm the current dates with both CAs, and find these cases during discovery before they break.

Why ECDSA where possible.

An ECDSA P-256 key gives roughly the same security as a 3072-bit RSA key, but handshakes are faster and certificates are smaller. Most modern clients support it. Keep RSA for systems that must talk to old clients, and record those systems: they will need attention again during the post-quantum migration.

How the two editions store policy

Both editions let you set policy once and reuse it, but they organise it differently.

- **Self-Hosted** uses the **Policy Tree**, a set of folders. A rule set on a folder applies to every certificate in that folder and all folders below it. A rule can also be **locked**, which means folders lower down cannot change it. For example, you can lock "allowed CAs = DigiCert, Sectigo" at the top folder for public certificates, and let each application folder below choose its own key size within the allowed range.
- **SaaS** uses **issuing templates**. An issuing template is a named set of rules (CA, key type, allowed names, validity and so on), and each **application** is given one or more templates. A field in a template can be fixed, so the requester cannot change it, or left open with a default value. Requests for that application can only use its templates.

In both cases the goal is the same: a person or pipeline requesting a certificate should never have to type a setting that the PKI team has already decided.

Layer your policy

Writing one policy per application does not scale, and one policy for the whole company is too blunt. Most companies use three layers, where each layer can only make the rules stricter, never looser:

- 1. Company baseline.** Rules that apply to everything: no keys smaller than RSA 2048, no SHA-1, only approved CAs, no key reuse.
- 2. Environment.** Different rules for internet-facing production, internal production, and non-production.
- 3. Application.** The names that application may request and who owns it.

An example of the environment layer for your company:

Environment	CA	Key	Maximum validity	Wildcards	Approval needed
Internet-facing production	DigiCert, Sectigo as backup	ECDSA P-256 or RSA 2048+	199 days (will drop with the public limit)	Only with PKI team approval	First request only; renewals automatic
Internal production	Private CA	ECDSA P-256	90 days	No	No
Non-production	Private CA	ECDSA P-256	30 days	Yes, within test domains	No

The short lifetimes for internal and non-production certificates are deliberate. They are not required by any outside rule, but they force those teams to automate now, so they are ready when public certificates reach 47 days.

Prevent first, detect second

The platform enforces policy at two moments:

- **At request time (prevent).** If a request breaks the policy, for example asking for a 1024-bit key or a domain the company does not own, the platform refuses it before it reaches DigiCert or Sectigo. This is cheap and immediate.
- **After discovery (detect).** Certificates found by discovery were not created through the platform, so nothing stopped them being weak. The platform compares each one against policy and reports violations: expired, weak key, unapproved CA, wildcard without approval, too long a lifetime. These go to the owner to fix, and the overall violation count becomes a compliance metric.

As more certificates move from monitored to managed, prevention covers more of the estate and the detection list shrinks.

Exceptions

Some systems cannot meet policy. An old appliance may only support RSA keys, or a partner may insist on a longer-lived certificate. Do not quietly loosen the policy for everyone. Instead, grant a written **exception** with four parts: what rule is broken, why, who owns the risk, and an end date when it will be reviewed. Exceptions without end dates become permanent by accident.

Who decides what

Governance means agreeing who is responsible for each decision. A common split:

Decision	PKI or certificate team	Application owners	Security architecture	Change management
Company baseline rules	Writes and maintains	Informed	Approves	Informed
Issuing templates and folders	Builds	Requests changes	Consulted	Informed
Exceptions	Records	Requests and owns the risk	Approves	Informed
Automated renewals in production	Runs the platform	Owens the outcome	Informed	Pre-approves as a standard change

The last row is worth arguing for early. If every automated renewal needs a change ticket and a meeting, automation stops being automatic. Most companies classify platform-driven renewals as a **standard change**: pre-approved once, then logged automatically each time.

IN YOUR ROLLOUT

- ▶ Is there a written company baseline, and is it enforced in the Policy Tree or issuing templates rather than only in a document?
- ▶ Which of your systems use public certificates for client authentication, and what is the plan for them?
- ▶ How many exceptions exist, and do they all have end dates?
- ▶ Are automated renewals approved as a standard change, or does each one need a ticket?

22.5

CA integrations: DigiCert, Sectigo and dual-CA failover

The platform does not sign certificates itself. It sends requests to your CAs, DigiCert and Sectigo, through their APIs and brings the signed certificates back. This section explains how that connection works, why domain validation is about to become the hardest part of it, and how to set up two CAs so that a problem at one does not stop your business.

What a CA integration does

A CA integration is a stored connection to one CA account. Through it, the platform can:

1. **Submit** a certificate request (a CSR plus the product type and names wanted).
2. **Wait and collect.** Some certificates are signed in seconds. Others, especially OV and EV, wait until the CA finishes its checks. The platform checks back (polls) until the certificate is ready.
3. **Revoke** a certificate when asked, with a reason.
4. **Import** the list of certificates already issued to the account, for the inventory (section 22.3).

In **Self-Hosted**, each connection is a **CA template** object in the Policy Tree. A CA template holds the account credentials and the product to order, so you may have several: for example, one for DigiCert OV single-name certificates and one for Sectigo OV multi-name certificates. Policy folders then point at the CA templates their certificates may use. In **SaaS**, you add each CA as a **CA account** (handled by a CA connector), and each issuing template names the CA account and product it uses.

DigiCert in practice

DigiCert's main customer platform is **CertCentral**. The pieces you will meet:

- **API key.** The platform authenticates with an API key created in CertCentral. A key acts with the permissions of the user or role it belongs to, so create it for a dedicated service user with only the rights it needs, and restrict it to the platform's IP addresses if your account allows.
- **Organizations.** For OV and EV certificates, DigiCert validates your company as an organization once. Certificate requests then reference that validated organization.
- **Domains.** Each domain (such as `example.com`) must be validated before DigiCert issues certificates for names under it. More on this below.

- **Products.** DigiCert sells several certificate products (single-name, multi-name, wildcard, OV, EV). The CA template or issuing template must name the exact product to order.
- **Plans.** Many companies buy **multi-year plans**: they pay for a period of coverage and receive a fresh certificate (a reissue) each time the shorter maximum lifetime runs out. With lifetimes shrinking, the platform will reissue under a plan several times a year without a new purchase. Check how your contract counts reissues.

DigiCert also sells its own lifecycle platform, Trust Lifecycle Manager. You will not need it if CyberArk is your platform, but you may see it in DigiCert documentation and sales conversations.

■ Sectigo in practice

Sectigo's customer platform is **Sectigo Certificate Manager (SCM)**. It used to be called Comodo Certificate Manager, which is why some CyberArk configuration fields still use the old "Comodo CCM" name. The pieces you will meet:

- **API credentials.** Depending on the API version, the platform uses an API user login, password and customer URI, or a client ID and client secret. Like the DigiCert key, these belong to a dedicated, least-privileged service account.
- **Organizations and departments.** SCM groups certificates and domain validations under organizations and departments. The integration must point at the right one, or requests will be rejected.
- **Certificate profiles.** The certificate product to order. In SCM version 20.5, Sectigo replaced its older "certificate types" with profiles, and some CA templates stopped working until administrators re-selected the product and validated the template again.

That last point is a general lesson, not a Sectigo quirk: **a change on the CA's side can break your integration without any change on yours.** Include a test issuance from every CA template in your regular health checks, so you notice a broken connection before a renewal depends on it.

■ Domain control validation: the part that is about to get hard

Before a public CA issues a certificate for `shop.example.com`, it must confirm that you control `example.com`. This is **domain control validation (DCV)**. The common methods are:

Method	How you prove control	Good for automation?
DNS TXT record	Publish a random value the CA gives you in a TXT record	Yes, if your DNS has an API the automation can use

Method	How you prove control	Good for automation?
DNS CNAME record	Publish a CNAME record pointing to a value the CA gives you	Yes, same condition
HTTP file	Place a file at a specific URL on the website	Partly: the website must already be reachable
Email	Click a link sent to an address such as <code>admin@example.com</code>	No: a human must click

Today, most companies validate each domain once in CertCentral and SCM and then issue for months without thinking about it, because the CA may reuse a validation for up to 200 days. Recall the table in section 22.1: that reuse period falls to 100 days in March 2027 and to **10 days in March 2029**. From then on, almost every renewal will need a fresh validation, so DCV must be automated through DNS.

There are two common ways to do this:

- **Give the automation access to your DNS provider's API**, so it can add and remove TXT records itself. This is simple but gives the automation power over your DNS, so scope the access tightly.
- **Delegate only the validation records**. Point a validation name (for example `_acme-challenge.example.com`, or the CA's equivalent) with a CNAME to a small separate DNS zone the automation controls. The automation can then answer validation challenges without being able to change any of your real DNS records.

The second approach is safer and worth proposing early in your project. DigiCert's 2024 mass revocation also shows why validation records matter: the bug was in how CNAME-based validation values were formed, and 83,267 certificates had to be replaced.

CAA records: the setting that silently blocks your backup CA

A **CAA record** is a DNS record that says which CAs are allowed to issue certificates for your domain. Public CAs must check it before issuing. For example:

```
example.com. CAA 0 issue "digicert.com"
example.com. CAA 0 issue "sectigo.com"
```

If your CAA records list only DigiCert, then Sectigo is **forbidden** from issuing for that domain, and your backup CA will fail at the exact moment you need it. Every domain you might issue from either CA must list both. Add an `iodef` entry too, so CAs can report blocked attempts to your security mailbox.

Dual-CA failover

Having two CAs only protects you if the second one is ready to take over. Four events in the last few years show why you want this:

- A CA must revoke your certificates at short notice (DigiCert, 2024).
- Browsers stop trusting a CA (Entrust, 2024).
- A CA's API is down or slow during a busy renewal window.
- A commercial dispute or price change makes you want to move volume quickly.

There are two ways to run two CAs:

Model	How it works	Strength	Weakness
Active-passive	DigiCert issues everything; Sectigo is configured and tested but idle	Simple to run and report on; one contract carries the volume	The backup path rots unless you test it
Active-active	Each CA issues for a defined share, for example by business unit or environment	Both paths are always proven to work	Two sets of products, contracts and support processes to manage

Whichever model you choose, failover only works if all of these are true:

- ☐ Both CAs are configured in the platform, and every issuing template or policy folder for public certificates can switch to the backup CA with one change.
- ☐ Every domain is validated (or can be validated automatically) at both CAs.
- ☐ CAA records allow both CAs.
- ☐ The organization is validated at both CAs for OV certificates.
- ☐ Applications do not depend on one CA's specific intermediate or root certificate. Some clients, especially older devices and partner systems, **pin** a specific intermediate or carry a short trust list. A certificate from the other CA has a different chain and will fail on those clients. Find them before a crisis.
- ☐ A test certificate is issued and installed from the backup CA at least once a quarter.
- ☐ A written runbook says who decides to fail over, which settings change, and how a mass renewal is started and watched.

Protect the CA credentials

The DigiCert API key and Sectigo credentials can buy certificates for your domains. An attacker holding them could get a trusted certificate for your website. Store them only in the platform's protected credential store (backed by an HSM if you have one, such as

Thales Luna), give them the smallest permissions that work, restrict them by IP address where the CA allows, rotate them on a schedule, and alert on any API use that does not come from the platform.

IN YOUR ROLLOUT

- ▶ Is Sectigo a tested backup or only a contract? When was a certificate last issued and installed from it?
- ▶ Do CAA records on every public domain allow both DigiCert and Sectigo?
- ▶ How is domain validation automated today, and will it still work when validation reuse falls to 10 days?
- ▶ Which clients or partners pin a DigiCert or Sectigo intermediate?
- ▶ Who can see or change the CA API credentials, and are they rotated?

22.6 Provisioning models and private key custody

Provisioning means getting the certificate, its chain and its private key onto the target and making the target use them. There are five ways to do it, and the right one depends on the target. The choice also decides who holds the private key, which is the security question that matters most.

The five provisioning models

Model 1: push through the target's API (agentless).

The connector runtime logs in to the target's management interface and installs the certificate itself. For example, it calls the F5 iControl REST API, the Panorama API, or the Akamai CPS API. Nothing is installed on the target. The platform needs network access to the target and an account with enough rights to change certificates. This is the main model for network devices and cloud services, and it is how most of your five targets are integrated.

Model 2: agent on the host (pull).

A small program runs on the server, checks in with the platform on a schedule, and installs certificates locally: in a Windows certificate store, a Java keystore, or a folder of PEM files used by Apache or nginx. Because the agent calls out, the platform does not need inbound access to the server. The cost is that you now have an agent fleet to install, update and monitor.

Model 3: the application asks for itself (API, CLI, SDK).

A build pipeline or deployment script calls the platform when it deploys, using the REST API or a tool such as CyberArk's VCert command-line tool. The request still has to follow the issuing template or policy, so the PKI team keeps control while the application team gets self-service. This suits teams that already automate everything through pipelines.

Model 4: a standard protocol.

The target runs a client for a standard enrollment protocol:

- **ACME** (RFC 8555), the protocol behind Let's Encrypt, used by web servers and Kubernetes.
- **EST** and **SCEP**, older protocols used by network devices and device management systems.

The platform, a CA, or a gateway acts as the server for that protocol. Check which protocols your edition and your CAs offer before designing around one.

Model 5: through another certificate system.

Some targets have their own certificate engine. Kubernetes and OpenShift use **cert-manager**, and HashiCorp **Vault** has a PKI secrets engine. Instead of replacing these, you connect them to the platform so their requests follow company policy:

- **cert-manager** has a **CyberArk issuer** (formerly called the Venafi issuer). One issuer points to one policy area (a "zone"), so certificates requested inside the cluster are checked and issued under that policy and appear in the inventory.
- **Vault** can use a plugin that sends its certificate requests to the platform, so applications keep using Vault's API while policy and inventory stay central. Chapter 23 compares this with letting Vault act as its own CA.

Comparing the models

Model	Best for	Who holds the key	What you must manage
1. Push via target API	Load balancers, firewalls, CDNs (F5, Panorama, Akamai)	The target, if it can make its own CSR; otherwise the platform creates the key and pushes it	Network reach and an admin-level service account on every target
2. Agent on host	Windows and Linux servers, Java applications	The host	Installing, updating and monitoring agents
3. API or CLI from the application	Teams with mature deployment pipelines	The application's environment	Pipeline credentials to the platform; making sure teams actually renew
4. Standard protocol (ACME, EST, SCEP)	Web servers, devices, anything with a built-in client	The target	The protocol server and how it maps requests to policy
5. Through cert-manager or Vault	OpenShift and Kubernetes; applications already using Vault	The cluster or Vault	Keeping the second system aligned with policy and inventory

Who creates the private key, revisited

Section 22.2 introduced the three places a key can be created. Here is what that means in practice for each model, and what to watch:

- **The target creates the key and a CSR.** Possible with F5 and Akamai CPS through their APIs, and natural with ACME, agents and cert-manager. The platform only ever sees the CSR and the certificate, which are not secret. This is the default to aim for.
- **The platform creates the key centrally, then pushes it.** Sometimes called **central key generation** or a **service-generated CSR**. It is needed when the target cannot

create a CSR through its API, or when several devices must share one key. The platform then stores the private key, and it must also send the key over the network. Insist on three protections: the key is stored encrypted (ideally with the encryption key held in an HSM such as Thales Luna), it is sent only over an encrypted management channel, and access to export it is limited and logged.

- **A person uploads a key and certificate they made themselves.** Sometimes unavoidable for legacy systems. Treat it as an exception (section 22.4): you have no proof of how the key was made or who else has a copy.

■ Clusters, pairs and many devices

Targets are rarely a single box. Three patterns need extra care:

- **High-availability pairs.** Two F5 devices in a sync group, or two firewalls in an HA pair, must present the same certificate. Either install on one and trigger the device's own sync, or install on both. Never leave the pair half-updated: if the device fails over, users get the old certificate.
- **Central managers.** Panorama pushes configuration to many firewalls. The platform installs on Panorama, but the certificate only reaches the firewalls when Panorama **commits and pushes**, which your network team may treat as a controlled change. Chapter 23 covers this.
- **Global propagation.** An Akamai certificate change rolls out across Akamai's edge network, which takes time. The platform must wait for deployment to finish before it can verify the change.

■ Credentials the platform needs for targets

Every push integration needs an account on the target. These accounts can change what your load balancers and firewalls present to the world, so treat them as privileged:

- Create one service account per target (or per group of targets), never a shared personal login.
- Give it only the rights to manage certificates and profiles, if the target supports such a role.
- Store its password or token only in the platform's credential store, or in a privileged access vault.
- Rotate it on a schedule and after anyone with access leaves.
- Alert on logins with that account that do not come from the connector runtime.

Choosing a model for each of your targets

A first view for your five targets (Chapter 23 gives the full playbook for each):

Target	Likely model	Key created where
Akamai	Push through the CPS API	Akamai, through a CPS enrollment that produces a CSR
F5 BIG-IP	Push through iControl REST	On the F5 where possible, otherwise pushed centrally
Palo Alto Panorama	Push through the Panorama API, then commit and push	Usually created centrally and imported
HashiCorp Vault	Through Vault, with Vault either requesting from the platform or acting as its own CA	Vault, or the application that asks Vault
OpenShift	Through cert-manager with the CyberArk issuer	Inside the cluster

IN YOUR ROLLOUT

- ▶ For each target, write down which of the five models it uses and where its private key is created.
- ▶ Which integrations use central key generation, and where are those keys stored afterwards?
- ▶ How are F5 HA pairs and Panorama commits handled, and what happens if the second half fails?
- ▶ Which service accounts on targets does the platform use, who else knows those passwords, and when were they last rotated?

22.7 Identity, access and approvals

The platform can obtain trusted certificates for any of your domains and install them on your most important systems. Anyone who controls it effectively controls your company's identity on the internet. So access must be split carefully: application teams get enough to serve themselves, the PKI team gets enough to run the platform, and nobody gets everything without someone else seeing it.

Who uses the platform

Five kinds of user, each with different needs:

User	What they need to do	What they must not be able to do
PKI administrators	Configure CAs, policy, templates, connectors and integrations	Silently issue certificates for any name without a record
Application owners	Request, renew and see the certificates for their own applications	See or change other teams' certificates; change policy
Approvers	Approve requests that policy marks as needing approval (for example, a new wildcard)	Approve their own requests
Auditors and security operations	Read the inventory, reports and audit log	Change anything
Machines (pipelines, cert-manager, scripts)	Request certificates through the API within one policy area	Use a person's account; reach policy areas they were not set up for

How access is set up in each edition

- In **Self-Hosted**, permissions are granted on **folders of the Policy Tree**. A user or group given rights on a folder has those rights on everything below it. A good design mirrors your organisation: one folder per business unit or application, with that team's group given rights on its own folder only.
- In **SaaS**, users belong to **Teams**. Teams are given access to **applications** and **issuing templates**. A team can only request certificates for its own applications, using the templates it has been given.

In both, connect the platform to your company's identity provider (Active Directory, Entra ID, Okta or similar) with single sign-on, and grant access through groups rather than to named people. When someone changes roles or leaves, removing them from the group removes their access, with no separate clean-up in the platform.

Machine identities for the platform itself

Pipelines, cert-manager and scripts also need credentials to call the platform. The rules mirror those for people:

- **One credential per system and purpose.** The OpenShift production cluster and the OpenShift test cluster each get their own credential, tied to their own policy area. If one leaks, you revoke one, not all.
- **Short-lived where possible.** SaaS uses API keys and supports token-based access; Self-Hosted uses OAuth tokens issued to registered API clients. Prefer short-lived tokens over long-lived keys, and rotate any long-lived key on a schedule.
- **Stored in a secrets system, not in code.** For example, in Vault or in Kubernetes secrets managed by your secret management process (Chapter 16).
- **Never a person's login.** When that person leaves, every pipeline using their login breaks, and in the meantime their personal account becomes a powerful target.

Approvals: where they help and where they hurt

An approval step means a second person must agree before a certificate is issued. It is a strong control, and it is easy to overuse.

Use approvals for requests that change your risk:

- The **first** certificate for a new application or a new domain.
- Wildcard certificates.
- Certificates for high-value names (login pages, payment pages, the company home page).
- Anything that needs a policy exception.

Do **not** require approval for routine renewals of a certificate that was already approved, with the same names and the same policy. If renewals wait for a human, you have rebuilt the manual process you were trying to escape, and at 47-day lifetimes the approvers will either drown or start approving without looking. The platform has already checked the renewal against policy. That check is the control.

A useful rule of thumb: **approve the first issue of a name, automate every renewal of it.**

Separation of duties

Separation of duties means no single person can complete a risky action alone. Applied to the platform:

- The people who configure CA connections and policy should not also be the only approvers of exceptions.
- Changes to the top-level policy, CA templates or CA accounts should need a second administrator to review, or at least should raise an alert that someone else sees.
- Exporting a private key from the platform should be rare, need a stated reason, and be logged and reported.
- Nobody should be able to delete or edit the audit log.

The audit log

The audit log records every action: who requested, approved, issued, installed, exported, revoked or changed what, and when. Three things make it useful rather than decorative:

1. **Send it to your security monitoring system** (your SIEM), where it is kept safe from tampering and can be searched alongside other logs.
2. **Alert on the events that matter:** a private key export, a change to CA credentials, a certificate issued outside any policy, a new administrator added, many revocations in a short time.
3. **Review a sample regularly.** A monthly look at who was granted administrator rights, and which exceptions were approved, catches drift before an auditor does.

IN YOUR ROLLOUT

- ▶ Is platform login through single sign-on, with access granted only through groups?
- ▶ Can any one person change CA credentials or top-level policy without a second person knowing?
- ▶ Do renewals of already-approved certificates still wait for a human anywhere?
- ▶ Does each pipeline and cluster have its own credential, scoped to its own policy area?
- ▶ Are key exports and CA credential changes alerting in your SIEM?

22.8 Renewal, rotation and revocation

Renewal is the job the platform does most often, and the one where small failures turn into outages. A safe renewal starts early, creates a new key, installs and verifies before touching the old certificate, and raises the alarm in steps if anything goes wrong. Revocation is the opposite job: making a certificate untrusted before it expires, which must be done in the right order or it causes the very outage you are trying to avoid.

When to renew

The platform starts a renewal when a certificate reaches its **renewal point**. There are two ways to set it:

- **A fixed number of days before expiry**, for example 30 days. Simple, but it breaks as lifetimes shrink: 30 days is plenty for a 398-day certificate and far too late for a 47-day one.
- **A percentage of the lifetime**, for example "when one third of the lifetime remains". This adapts automatically: about 66 days before expiry for a 200-day certificate, 33 days for a 100-day one, and about 16 days for a 47-day one.

Use the percentage approach. It keeps a consistent safety margin no matter what lifetime a certificate has.

Some CAs also publish a suggested renewal window through **ACME Renewal Information (ARI)**, an extension of the ACME protocol. If a CA knows it will need to revoke certificates early (as DigiCert did in 2024), it can bring the suggested window forward, and clients that follow ARI renew early without any human action. Ask whether your CAs and platform support it.

What can go wrong, step by step

A renewal passes through several steps, and each can fail for its own reasons. Knowing them helps you read an error and know whom to call.

Step	Typical failure	What it looks like	Usual fix
Request	The request breaks policy (a name not allowed, a key too small)	Rejected by the platform before reaching the CA	Fix the template or request; agree an exception if needed

Step	Typical failure	What it looks like	Usual fix
Domain validation	The DNS record could not be published, or the CA could not see it	CA order stuck as "pending validation"	Check the DNS automation and delegation (section 22.5)
Issuance	CA API down, rate limit hit, product not on contract	Error from the CA connector	Retry; fail over to the other CA if it persists
Install	Service account password expired, device unreachable, target API changed after an upgrade	Connector error on the installation	Fix credentials or network; check target version notes
Bind	New certificate installed but the profile still points to the old one	Install succeeds, verify fails	Check the profile name mapping in the integration
Sync	One half of an HA pair or one cluster node not updated	Works on one device, old certificate on the other	Trigger sync; add a check on both nodes
Verify	The service still serves the old certificate, or the chain is incomplete	Verification error	Look at caching, load balancer pools, CDN propagation

■ The escalation ladder

A failed renewal is not yet an outage, because the old certificate is still valid. That gives you time, and the platform should use it in steps. An example for a 200-day certificate that starts renewing about 66 days before expiry:

Days before expiry	What happens	Who is told
66	Renewal starts automatically	Nobody, unless it fails
60 to 30	Failed steps retry automatically	Certificate owner, by email or ticket
30	Still failing	Owner and owner's manager; ticket raised
14	Still failing	PKI team on call takes the ticket
7	Still failing	Raised as an incident with the service's support team

Scale the numbers to your lifetimes. The idea is that every certificate has an owner, each step widens the circle, and no certificate reaches its expiry date with only an unread email as warning.

■ Install first, verify, then let the old one go

The safe order for every renewal:

1. Get the new certificate.
2. Install it alongside the old one, if the target allows two at once.
3. Switch the binding to the new certificate.
4. **Verify from the outside:** connect as a client would and check that the new certificate is served, the full chain is sent, the names match, and the dates are correct.
5. Only then remove the old certificate from the target, or leave it to expire.

If verification fails, switch the binding back. The old certificate is still valid, so rolling back is safe. This is why renewing early matters: it leaves time to roll back and try again.

■ Key rotation: a new key every time

Every renewal should create a **new key pair**. If an old private key was copied without anyone knowing, a new key ends the attacker's access at the next renewal. With short lifetimes and new keys, the damage from an unnoticed key leak is limited to weeks instead of years.

The usual reason teams resist new keys is **pinning**: an application or partner that trusts one specific public key rather than the CA. Pinning breaks whenever the key changes. Find pinned systems during discovery, ask the owners to pin to the CA's intermediate or root instead (or better, not pin at all), and keep them on a written exception until they do.

■ Revocation: what it is and when to use it

Revocation tells clients to stop trusting a certificate before its expiry date. You ask the CA to revoke, the CA publishes this, and clients that check will refuse the certificate. Reasons to revoke:

Situation	Revocation reason to give the CA	Urgency
The private key was or might have been exposed	Key compromise	Highest: act within hours
The certificate was replaced and the old one is no longer used	Superseded	Low
The service was shut down	Cessation of operation	Low
Information in the certificate is no longer true (company name, domain sold)	Affiliation changed	Medium
The CA tells you it must revoke (as in the DigiCert event)	Set by the CA	Set by the CA, often 24 hours to 5 days

One warning about relying on revocation: many clients check revocation poorly or not at all, for reasons covered in Chapter 9. Revocation is still required, but it is not a guarantee. **Short lifetimes are the stronger protection**, because a stolen key for a 47-day certificate is useless within weeks even if nobody checks revocation.

■ The order that prevents self-inflicted outages

Revoking a certificate that is still in use takes the service down for every client that checks revocation. So the order is always:

1. **Replace** the certificate on every installation (the inventory tells you where they all are).
2. **Verify** that every installation serves the new certificate.
3. **Revoke** the old one.

With a key compromise, the same order applies, only faster. The platform's value here is speed: because it knows every installation and can renew all of them automatically, step 1 takes minutes instead of days, and step 3 can follow the same hour.

■ Mass replacement

Sometimes many certificates must be replaced at once: a CA revocation event, a CA being distrusted, a weak key type being banned, a compromised intermediate. The platform turns this into a repeatable procedure:

1. **Select** the affected certificates with a query: by issuing CA, by intermediate, by key type, by domain, or by issue date.
2. **Prioritise**: internet-facing and revenue-critical services first.
3. **Change the policy** if the cause requires it, for example switching the CA to Sectigo for the affected folders or templates.
4. **Renew in batches**, watching the CA's rate limits and the error rate.
5. **Track to zero**: a live report of affected certificates still in use, reviewed until it reaches zero.
6. **Chase the leftovers**: the certificates that could not be automated need owners to act by hand. Those are the ones that cause the outage, so start on them first.

Practise this once a year with a harmless test, such as replacing all non-production certificates from one CA. Teams that have practised recover in hours; teams that have not take days.

IN YOUR ROLLOUT

- ▶ Are renewal points set as a percentage of lifetime or as a fixed number of days?
- ▶ Does every integration verify from the outside after installing?
- ▶ What does the escalation ladder look like today, and does the final step reach a person who can act?
- ▶ Which systems pin keys, and what are their exceptions?
- ▶ Could you produce, in five minutes, a list of every installation of certificates from one DigiCert intermediate?

22.9

The rollout program and the metrics that prove it works

Buying and installing the platform is the easy part. The hard part is changing how hundreds of teams get and look after certificates. Treat the rollout as a program with four phases, a clear gate before each new phase, a planned order for onboarding targets, and a small set of numbers that show whether it is working.

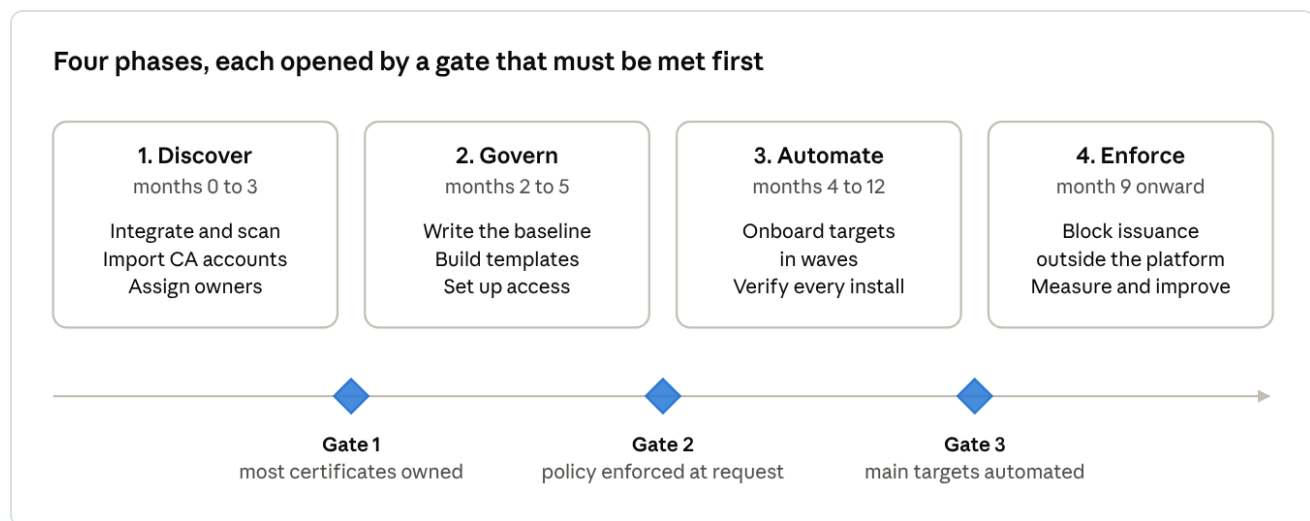


Figure 22.2 An example 12-month rollout in four phases. Each gate must be met before the next phase depends on it.

The phases overlap on purpose: you start writing policy while discovery is still finishing. The gates are what keep the overlap honest. Each one is a condition that must be true before the next phase is allowed to depend on the last. The months are an example for a large company; yours depend on the size of your estate and team.

Phase 1: Discover

Goal: know what you have and who owns it.

- Connect integration discovery to Akamai, F5, Panorama, Vault and OpenShift.
- Import the DigiCert and Sectigo account histories.
- Turn on Certificate Transparency monitoring for every public domain.
- Run network scans, with hostnames, in each network zone.
- Assign owners and start working the unowned queue (section 22.3).

Gate 1: most certificates, and every internet-facing one, have a named owner. Without owners, the next phases have nobody to alert, approve or fix anything.

Phase 2: Govern

Goal: decide the rules and put them in the platform.

- Write the company baseline and the environment rules (section 22.4).
- Build the Policy Tree folders or SaaS applications and issuing templates.
- Connect single sign-on and set up access by group (section 22.7).
- Agree with change management that platform renewals are a standard change.
- Configure DigiCert and Sectigo, CAA records and DNS validation (section 22.5).

Gate 2: a new certificate requested through the platform is checked against policy before it reaches a CA, and the CAA records allow both CAs.

Phase 3: Automate

Goal: move certificates from **monitored** to **managed**, one wave of targets at a time.

Onboard targets in waves, so each lesson is learned once. A sensible order for your estate:

Wave	Targets	Why in this order
1	F5 BIG-IP and Akamai, internet-facing first	These carry customer traffic, so an expiry hurts most, and both have mature APIs
2	OpenShift through cert-manager	Large numbers of short-lived certificates; once the issuer works, teams serve themselves
3	HashiCorp Vault	Needs an architecture decision first: is Vault a CA, or a client of the platform? (Chapter 23)
4	Palo Alto Panorama	Commits and pushes are controlled changes, so agree the process with the network team before automating
5	Everything else found in discovery	Servers with agents, then whatever is left, with exceptions for systems that cannot be automated

For each wave, run the same steps: pilot on a few non-production certificates, then a few production ones with the owner watching, then the rest. Check every installation with the verify step. Write down what failed and fix the integration before the next batch.

Gate 3: the main targets renew automatically, with verification, and failures follow the escalation ladder (section 22.8) without anyone having to watch.

Phase 4: Enforce

Goal: make the platform the only way to get a certificate.

- Remove or restrict direct access to the DigiCert and Sectigo portals, so teams cannot buy certificates around the platform.
- Use Certificate Transparency monitoring to catch any public certificate for your domains that did not come through the platform, and follow each one up.
- Shorten internal lifetimes step by step, to prove readiness for 100-day and 47-day public limits.
- Review metrics monthly and fix the biggest gap each month.

Who does what in the program

Role	Responsibilities
PKI or certificate platform team	Runs the platform, CA connections and integrations; owns policy and the escalation ladder
Application owners	Own their certificates; respond to alerts; help pilot their own integrations
Network and load balancer team	Service accounts and API access on F5 and Panorama; HA and commit processes
Cloud and platform engineering	OpenShift, cert-manager and Vault integrations
Security architecture and risk	Approve baseline policy and exceptions; read the reports
Change management	Agree standard-change treatment for automated renewals

The numbers that show it is working

Pick a few metrics, publish them every month, and show the trend. These are the ones that matter most:

Metric	How it is calculated	Why it matters
Ownership coverage	Certificates with a named owner, divided by all certificates	You cannot fix what nobody owns
Automation coverage	Managed certificates, divided by all certificates	The share that will survive 47-day lifetimes
Policy compliance	Certificates with no policy violation, divided by all certificates	Measures risk in the estate
Renewal success rate	Renewals that finished without a human, divided by all renewals	Shows whether automation is reliable
Days of warning	For failed renewals, how many days before expiry a person was alerted	Shows the escalation ladder working

Metric	How it is calculated	Why it matters
Expiry incidents	Outages caused by expired certificates	The number leadership cares about. It should reach zero
Out-of-platform issuance	Public certificates found in CT logs that did not come through the platform	Shows whether enforcement is real

Report certificates and installations separately (section 22.3), and always show public and private certificates apart, because their rules and risks differ.

■ Pitfalls that slow rollouts down

- **Treating it as a tool install.** The platform works only when owners respond, policy is agreed and change management cooperates.
- **Automating before owners exist.** Alerts go nowhere, and failures turn into outages anyway.
- **Approval on every renewal.** It feels safe and quietly defeats the purpose (section 22.7).
- **Install without verify.** The platform reports success while users still get the old certificate.
- **A backup CA that was never tested.** It fails on the day you need it, often because of CAA records (section 22.5).
- **Counting the easy wins.** Hundreds of OpenShift certificates make the automation number look great while the twenty manual certificates on legacy systems are the ones that will cause the next outage. Track the leftovers by name.

IN YOUR ROLLOUT

- ▶ Which phase is your program in for each target, and has each gate actually been met?
- ▶ Which of the seven metrics are you reporting today, and to whom?
- ▶ Is there a named list of certificates that cannot be automated, with owners and plans?
- ▶ Can teams still buy certificates directly from DigiCert or Sectigo without going through the platform?

HANDS-ON LAB

Lab: a miniature certificate platform on your laptop

This lab rebuilds the core jobs of a certificate lifecycle platform with free tools, so you can see each job working with your own eyes: a private CA, policy that refuses bad requests, discovery that misses certificates unless it sends hostnames, an inventory, renewal, and revocation. Nothing here touches your company's systems. Allow about three hours.

What you need

- A laptop with **Docker** installed.
- **OpenSSL** (already on macOS and Linux; on Windows use WSL).
- The **step** command-line tool from Smallstep (install instructions at smallstep.com/docs/step-cli).

You will use **step-ca**, a free private CA from Smallstep, as a stand-in for DigiCert or Sectigo, and OpenSSL's built-in test server as a stand-in for an F5 or a web server.

Lab part	Platform job it imitates	Section
A. Start a private CA	The CA layer	22.2, 22.5
B. Add a naming policy	Policy enforced at request time	22.4
C. Issue and serve certificates	Issuance and provisioning	22.6
D. Discover them, with and without SNI	Network discovery	22.3
E. Build a small inventory	Inventory records	22.3
F. Renew and verify	Renewal with verification	22.8
G. Revoke in the right order	Revocation	22.8

Part A: start a private CA

Create a working folder, then start step-ca in Docker. The environment variables tell the container to set itself up on first start.

```
mkdir clm-lab && cd clm-lab
docker volume create stepca
docker run -d --name stepca -p 9000:9000 -v stepca:/home/step \
  -e "DOCKER_STEP_CA_INIT_NAME=Lab CA" \
  -e "DOCKER_STEP_CA_INIT_DNS_NAMES=localhost,stepca" \
  smallstep/step-ca
```

Read the container log to find the administrator password the CA created for itself:

```
docker logs stepca
```

Look for a line saying your CA administrative password is a long random string. Save it in a file called `password.txt` in your lab folder.

Now get the **fingerprint** of the CA's root certificate. A fingerprint is a short hash that uniquely identifies a certificate. Your laptop will use it to make sure it is talking to the right CA the first time it connects.

```
docker exec stepca step certificate fingerprint certs/root_ca.crt
step ca bootstrap --ca-url https://localhost:9000 --fingerprint <paste-fingerprint-here>
```

What just happened: you now have a root and an intermediate CA, like a tiny DigiCert. The `bootstrap` command downloaded the root certificate and told your `step` tool to trust it.

Part B: add a naming policy

A real platform refuses requests for names your company does not own. `step-ca` can do the same. Open the CA's configuration file inside the container:

```
docker exec -it stepca vi config/ca.json
```

Find the `"authority"` section and add a `policy` block inside it, so it looks like this (keep everything else that is already there):

```
"authority": {
  "policy": {
    "x509": {
      "allow": { "dns": ["*.lab.internal"] }
    }
  },
  "provisioners": [ ... leave as it was ... ]
}
```

Save, then restart the CA with `docker restart stepca`.

What just happened: the CA will now only sign certificates for names ending in `.lab.internal`. You will test this in the next part. In CyberArk, the same rule would live in a locked policy setting or an issuing template (section 22.4).

Part C: issue certificates and serve them

Request two certificates with a short life of two hours, so you can watch them age during the lab:

```
step ca certificate web1.lab.internal web1.crt web1.key --not-after 2h --provisioner-
password-file password.txt
step ca certificate web2.lab.internal web2.crt web2.key --not-after 2h --provisioner-
password-file password.txt
```

Now try a name the policy does not allow:

```
step ca certificate shop.example.com bad.crt bad.key --provisioner-password-file
password.txt
```

This should be **refused**. That refusal is "prevent" from section 22.4: the bad certificate never existed.

Next, start one test server that holds **both** certificates on the **same port**, choosing between them by the name the client asks for (SNI). This imitates a load balancer hosting two websites on one IP address:


```
openssl s_server -accept 8443 -www \  
-cert web1.crt -key web1.key \  
-servername web2.lab.internal -cert2 web2.crt -key2 web2.key
```

Leave this running and open a second terminal in the same folder.

■ Part D: discover, with and without SNI

First, scan the way a naive scanner does, with an address and port but no hostname:

```
echo | openssl s_client -connect localhost:8443 2>/dev/null \  
| openssl x509 -noout -subject -enddate
```

You see **only** `web1.lab.internal`, the default certificate. Now ask for each hostname:

```
for host in web1.lab.internal web2.lab.internal; do  
    echo | openssl s_client -connect localhost:8443 -servername $host 2>/dev/null \  
    | openssl x509 -noout -subject -enddate  
done
```

Now both certificates appear.

What just happened: this is the SNI problem from section 22.3 in miniature. A scan that sends only IP addresses would have missed half of this server's certificates. On a real F5 or Akamai configuration with hundreds of sites, it would miss most of them.

■ Part E: build a small inventory

Write each discovered certificate to a CSV file, one row per installation, with the fingerprint that identifies the certificate:

```

echo "host,port,servername,subject,not_after,sha256_fingerprint" > inventory.csv
for host in web1.lab.internal web2.lab.internal; do
    cert=$(echo | openssl s_client -connect localhost:8443 -servername $host 2>/dev/null)
    subject=$(echo "$cert" | openssl x509 -noout -subject | sed 's/subject=//')
    end=$(echo "$cert" | openssl x509 -noout -enddate | sed 's/notAfter=//')
    fp=$(echo "$cert" | openssl x509 -noout -fingerprint -sha256 | sed 's/.*=//')
    echo "localhost,8443,$host,\"$subject\", \"$end\", $fp" >> inventory.csv
done
cat inventory.csv

```

Now answer three questions from the file, the same ones a platform answers: How many unique certificates are there (count distinct fingerprints)? How many installations? Which expires first? Then add an **owner** column by hand. Notice that nothing in the certificate tells you the owner. That is why ownership is the hardest part of section 22.3.

Part F: renew and verify

Renew `web1` with a new key, as policy requires:

```
step ca renew --force --rekey web1.crt web1.key
```

The server in the first terminal still has the old certificate loaded in memory. Run your discovery loop again, and you will see the **old** expiry date for `web1`. This is the "installed but not bound" failure from section 22.8: the new files exist, but the service has not picked them up.

Stop the server with Ctrl+C, start it again with the same command, and repeat the discovery loop. Now the new expiry date appears. That second check is the **verify** step, and it is the only proof users are getting the new certificate.

Finally, try a renewal while the CA is down, to see a failure the escalation ladder must catch:

```

docker stop stepca
step ca renew --force --rekey web2.crt web2.key
docker start stepca

```

The renewal fails, but `web2` keeps working because its current certificate is still valid. Renewing early is what turns this from an outage into a ticket.

Part G: revoke in the right order

Pretend `web2`'s key leaked. Follow the order from section 22.8: replace, verify, then revoke.

```
cp web2.crt web2-old.crt
step ca certificate web2.lab.internal web2.crt web2.key --not-after 2h --provisioner-
password-file password.txt
# restart the openssl s_server from Part C, then run the discovery loop to verify
step ca revoke --cert web2-old.crt --key web2.key
```

The last command will fail, because `web2.key` is now the **new** key. Revocation by certificate and key needs the **old** key, which you overwrote. Look up the serial number of the old certificate and revoke by serial instead:

```
step certificate inspect web2-old.crt --format json | grep serial_number
step ca revoke <serial-number> --provisioner-password-file password.txt
```

What just happened: you met a real operational lesson. When you replace a compromised certificate, record its serial number before anything is overwritten. Platforms do this for you, which is one quiet reason they exist.

Clean up

```
docker rm -f stepca && docker volume rm stepca
```

Check your understanding

1. In Part D, which discovery method from section 22.3 did you imitate, and which other method would have found `web2` even without SNI?
2. In Part F, the renewal "succeeded" but users would still see the old certificate. Which step of the renewal pipeline failed?
3. In Part B, would the policy have stopped someone who bought a certificate for `web1.lab.internal` from a different CA? What would catch that in real life?

Answers: (1) Network scanning; integration discovery reading the server's configuration would have listed both certificates. (2) Bind, caught by verify. (3) No. Policy only governs requests that go through your platform. Certificate Transparency monitoring catches public certificates from other CAs.

TOOLKIT

At work: checklists and templates to copy

These five tools turn the chapter into things you can use this week. Copy them into your team's wiki and adjust the details to your environment.

1. New integration onboarding checklist

Use this every time a new target type (or a new instance of one) is connected to the platform.

Before building

- ☐ Target type, version and owner team are recorded.
- ☐ Provisioning model chosen (section 22.6) and written down.
- ☐ Where the private key is created is decided, and the security team agrees.
- ☐ A dedicated service account exists on the target, with only the rights needed for certificates and profiles.
- ☐ Service account credentials are stored only in the platform credential store or privileged access vault.
- ☐ Network path from the connector runtime (VSatellite or driver host) to the target is open and documented.
- ☐ HA pairs, sync groups or central managers (Panorama, F5 device groups) are identified.
- ☐ Change management agrees how automated installs on this target are classified.

During the pilot

- ☐ One non-production certificate renewed end to end, including bind and verify.
- ☐ A failure is forced on purpose (wrong password, unreachable device) and the alert reaches the owner.
- ☐ Rollback tested: the old certificate is put back and verified.
- ☐ One production certificate renewed with the owner watching.

Before calling it done

- ☐ Every installation on the target is in the inventory and linked to its certificate.
- ☐ Renewal point and escalation ladder are set.
- ☐ Runbook written for the target's common failures.
- ☐ Service account rotation is scheduled.

2. Certificate request intake form

When an application team asks for a new certificate (not a renewal), collect this once. After that, renewals need nothing from them.

Field	Example	Why it is asked
Application name and CMDB ID	Payments API, APP-01234	Links the certificate to an owner and a service
Owner group (not a person)	team-payments	Alerts still reach someone when people move
Escalation contact	payments-oncall	Used at the top of the escalation ladder
Names needed	api.pay.example.com	Checked against allowed domains
Public or private	Public	Decides CA and policy layer
Environment	Production	Decides lifetime and approval rules
Where it will be installed	F5 pair lb-dc1-01 and lb-dc1-02, profile clientssl-payapi	Lets the platform automate install and verify
Client authentication needed?	No	Public certificates are dropping client authentication (section 22.4)
Any pinning or partner trust lists?	Partner X trusts DigiCert only	Affects key rotation and CA failover

3. Renewal failure triage runbook

When a renewal alert arrives, work through these questions in order. Each one maps to a step in section 22.8.

- 1. How long is left?** Check the current certificate's expiry. More than 14 days: normal ticket. Fewer than 7: treat as an incident.
- 2. Which step failed?** Read the platform's job history: request, validation, issuance, install, bind, sync or verify.
- 3. Request or policy failure?** The request broke a rule. Fix the request or the template; do not loosen policy to get past it.
- 4. Validation failure?** Check the DNS validation record exists and is visible from the internet, and that CAA allows the CA.
- 5. Issuance failure?** Check the CA's status page and the platform's CA connector test. If the CA is down and time is short, switch that certificate to the backup CA.
- 6. Install failure?** Check the service account (expired password, locked account), network reachability from the connector runtime, and whether the target was recently

upgraded.

7. **Bind, sync or verify failure?** Log in to the target and check which certificate the profile, route or service actually uses, on every node.
8. **Fixed?** Rerun the renewal, confirm verification passes, and record the cause so the integration can be improved.

4. Recurring operations checklist

Weekly

- ☐ Work the unowned certificate queue.
- ☐ Review all failed renewals and their causes.
- ☐ Review certificates found in CT logs that did not come through the platform.

Monthly

- ☐ Publish the seven program metrics (section 22.9).
- ☐ Review new administrators, key exports and CA credential changes in the audit log.
- ☐ Review exceptions that end this month.

Quarterly

- ☐ Issue and install a test certificate from the backup CA.
- ☐ Test every CA template or CA account connection.
- ☐ Rotate service account and CA API credentials that are due.
- ☐ Check CAA records on all public domains still list both CAs.

Yearly

- ☐ Run a mass replacement exercise on non-production certificates.
- ☐ Review the company baseline against new CA/Browser Forum and CA changes.

5. Questions to ask in any design review

Bring these to any meeting about a new integration, a new application, or a change to the platform. Each one exposes a gap that has caused real outages or audit findings.

1. Where is each private key created, and who can read it afterwards?
2. Which system is the source of truth for this certificate's inventory record?
3. What happens if the renewal succeeds in the platform but the install fails on the target?
4. How is the new certificate verified after install, and who is alerted if verification fails?

5. Is the renewal point sized for the shortest lifetime this certificate will have by 2029?
6. Who owns this certificate, and what happens when that person leaves?
7. Could the target keep up with renewals every month?
8. If we had to move this certificate to the other CA tomorrow, what would break?
9. Does anything pin this certificate's key or its CA?
10. Will this certificate need a post-quantum plan, and where is that recorded?

CASE STUDY

Failure story: the expired certificate behind the Equifax breach

In 2017, attackers stole personal data on about 147 million people from Equifax, one of the three large US credit agencies. Most coverage focused on an unpatched web application flaw that let the attackers in. Less attention went to why nobody noticed them for 76 days. The answer was an expired certificate, and every control in this chapter would have caught it.

What happened

Equifax ran a network monitoring device that watched traffic leaving its online dispute portal (the system consumers used to dispute credit report errors). Because that traffic was encrypted, the device needed a certificate to decrypt and inspect it.

That certificate had **expired 19 months earlier**. With an expired certificate, the device could not inspect the traffic, so it was effectively switched off. Nobody was alerted, because nothing "broke" in a way users would notice. The website kept working. Only the security control had stopped.

Attackers entered on 13 May 2017. On 29 July, an employee updated the expired certificate. Almost immediately, the device showed suspicious traffic to an address in China, and the investigation began. By then, data had been leaving for 76 days.

The US House Oversight Committee's report on the breach found that Equifax had let **more than 300 certificates expire**, including 79 used to monitor business-critical domains. It concluded that the breach was entirely preventable, and pointed to unclear responsibility in IT management as a root cause. In its words, a certificate management process with defined roles and responsibilities would have kept the monitoring device's certificate active when the intrusion began.

What would have caught it

What went wrong	Control from this chapter	Section
Nobody knew the certificate was there or what depended on it	Integration discovery of the monitoring device; inventory linking certificate to installation	22.3
Nobody owned it	Mandatory owner assignment and an unowned queue worked weekly	22.3

What went wrong	Control from this chapter	Section
It expired silently	Renewal point at a percentage of lifetime; automated renewal	22.8
When renewal did not happen, nobody was told	Escalation ladder ending in an incident before expiry	22.8
More than 300 others expired too	Policy compliance reporting; expired certificates as a monthly metric	22.4, 22.9
Responsibility was unclear	Governance table of who decides and who acts	22.4, 22.9

■ The lessons to carry into your rollout

- 1. Certificates on security tools matter as much as those on websites.** Network monitoring, TLS inspection, proxies and logging systems often use certificates nobody thinks about, and when they expire, protection fails silently instead of loudly. Make sure discovery includes them, and give them owners.
- 2. An outage is the lucky failure.** When a website certificate expires, customers complain within minutes. When a monitoring certificate expires, nobody complains. Count "expired certificates in use" as a metric, not only "outages".
- 3. Ownership is the control that makes all the others work.** Equifax's problem was not a missing tool. It was that no one was clearly responsible. Gate 1 of the rollout (section 22.9) exists for exactly this reason.

KEY TAKEAWAYS

Chapter summary

If you remember one sentence from this chapter: **a certificate lifecycle platform exists so that every certificate has an owner, follows a rule, renews itself early with a new key, and is checked from the outside after every change.**

The rest, in brief:

- **Why (22.1).** Expired certificates cause outages and silently disable security tools. Public certificate lifetimes fall to 100 days in March 2027 and 47 days in March 2029, which makes manual renewal impossible at scale.
- **Architecture (22.2).** Four layers: CAs, the control plane, a connector runtime inside your network, and the targets. CyberArk Certificate Manager comes as Self-Hosted (Policy Tree, CA templates, drivers) or SaaS (applications, issuing templates, VSatellites), now also sold by Palo Alto Networks as Next-Gen Trust Security.
- **Discovery (22.3).** Combine integration discovery, CA import, CT monitoring, network scans with hostnames, and host agents. Separate certificates from installations. Ownership is the hardest and most important step.
- **Policy (22.4).** Prevent at request time, detect after discovery. Layer the rules, block wildcards by default, keep exceptions dated, and treat automated renewals as standard changes.
- **CAs (22.5).** DigiCert and Sectigo connect through their APIs. Domain validation must be automated through DNS before reuse falls to 10 days. CAA records must allow both CAs, or the backup will fail.
- **Provisioning (22.6).** Five models. The key question is always where the private key is created. Aim for the target.
- **Access (22.7).** Single sign-on and groups for people, one scoped credential per machine, approvals on first issue only, and an audit log in your SIEM.
- **Renewal and revocation (22.8).** Renew at a percentage of lifetime, install then verify then retire, escalate in steps. Replace before you revoke.
- **Program (22.9).** Discover, govern, automate, enforce, with a gate before each phase and seven metrics to prove progress.

CHECK YOURSELF

Review questions

Try to answer each one in two or three sentences without looking back. The section to check is in brackets.

1. Name the four reasons companies buy a certificate lifecycle platform, with one real example for any two of them. (22.1)
2. Why does the shrinking domain validation reuse period matter as much as the shrinking certificate lifetime? (22.1, 22.5)
3. In Self-Hosted, what is an "Application" object? In SaaS, what is an "Application"? Why does this cause confusion? (22.2)
4. What does a VSatellite do, and why does it connect outwards to the cloud rather than the cloud connecting in? (22.2)
5. Give two reasons network scanning alone gives an incomplete inventory. (22.3)
6. A dashboard says you have 12,000 certificates. What question should you ask before trusting the number? (22.3)
7. What is the difference between preventing and detecting a policy violation? Give an example of each. (22.4)
8. Why might a system that uses a public TLS certificate for client authentication break in 2027? (22.4)
9. Your DigiCert account is suspended for a day. List three things that must already be true for Sectigo to take over. (22.5)
10. Describe the three places a private key can be created, and rank them by risk. (22.6)
11. Why should routine renewals not need approval, and what control replaces the approval? (22.7)
12. A renewal installs the new certificate but users still see the old one. Which step failed, and how would the platform have noticed? (22.8)
13. Explain why revoking before replacing causes an outage, and give the correct order. (22.8)
14. What is Gate 1 of the rollout, and why must it come before automation? (22.9)
15. In the Equifax breach, why did nobody notice the expired certificate for 19 months? (Failure story)

REFERENCES

Sources

Product names, dates and vendor features change often. These are the pages this chapter relied on, checked in October 2026 through search results. Product details (VSatellite behaviour, connector names, key generation options) should be confirmed in your own edition's documentation before you rely on them in a design.

CyberArk and Palo Alto Networks

- [CyberArk Certificate Manager, Self-Hosted datasheet](#)
- [CyberArk Certificate Manager, SaaS datasheet](#)
- [CyberArk Certificate Manager, SaaS: issuing templates and policy](#)
- [Certificate Manager, Self-Hosted introduction \(docs.venafi.com\)](#)
- [Self-Hosted: CA integration setup](#)
- [Self-Hosted: Adaptable driver overview](#)
- [Self-Hosted: Sectigo Certificate Manager CA attributes](#)
- [VSatellites overview \(Palo Alto Networks Next-Gen Trust Security docs\)](#)
- [CyberArk Certificate Manager is now Next-Gen Trust Security](#)
- [Palo Alto Networks completes CyberArk acquisition \(CRN Asia\)](#)
- [cert-manager CyberArk issuer documentation](#)

CA/Browser Forum and certificate lifetimes

- [CA/Browser Forum Ballot SC-081v3](#) (the primary source; the schedule in 22.1 was cross-checked against summaries such as [this timeline](#))
- [Sectigo: client authentication ECU deprecation FAQ](#)
- [GeoCerts change log \(DigiCert client authentication timeline\)](#)

DigiCert and Sectigo

- [DigiCert to revoke 83,000 certificates \(The Hacker News\)](#)
- [DigiCert revoking certificates of 6,800 customers \(SecurityWeek\)](#)
- [Sectigo SCM: understanding CA connectors](#)
- [SCM 20.5 certificate profile change affecting Venafi CA templates](#)

Equifax

- [House Oversight Committee report coverage \(CyberScoop\)](#)

- [Equifax breach report highlights multiple security failures \(TechTarget\)](#)